

制御構造

授業：コンピュータ基礎

元資料：コンピュータ基礎/pdf/制御構造.pdf

生成元：local

【要約】

制御構造とプログラムの図示 原田研究室 准教授 万 偉偉 基礎工学部 F522号室 wan@sys.jp 復習・アルゴリズムとは・問題を解くための論理または手順・プログラミングとは・データ構造とアルゴリズム・良いプログラム、計算方法の選択・事前条件・事後条件・不変条件・自然言語・疑似コーディング・Cプログラム・データ型・文字・整数・実数・配列・構造体・共用体・ポインタ・データ構造・リスト、待ち行列・スタック・木・ハッシュ表・グラフ・データ構造+操作の手法 → 抽象データ型・演算子・優先順位 本日の内容・フローチャート・NSチャート・プログラムの制御構造（構造化プログラミング）・擬似言語（擬似コード、PDL）・拡張バックス記法・ハイレベル 計算機言語の制御構造・接続構造・選択構造・単一選択構造(if ~ then ~ end if)・分岐選択構造(if ~ then ~ else ~ end if)・複数選択構造(switch ~ end switch)・反復構造・前判定反復構造(while ~ end while)・後判定反復構造(do ~ until)・カウンタ制御反復構造(for) 各制御構造の図示・フローチャート（流れ図, Flow Chart）・論文や説明書提案書の作成によく使われる・NSチャート（Nassi-Schneiderman Chart） 接続構造・処理1, 処理2…処理nの逐次実行・このようなまとまりをブロック(block)という。 処理1 処理2 処理n 流れ図 処理1 処理2 処理n NSチャート 単一選択構造・条件が真のときに処理を行う・C言語: if (条件) 処理; 流れ図 条件 真 NSチャート 条件 真 偽 処理 偽 分岐選択構造・C言語: if (条件) then 処理1; else 処理2; 流れ図 NSチャート 条件 真 偽 処理1 処理2 条件 真 処理1 偽 処理2 複数選択構造・C言語: switch (条件) { case 値1: 処理1; ... }; 流れ図 条件=値2 真 処理1 処理2 真 条件=値1 偽 条件=値n 処理n 真 偽 偽 複数選択構造・C言語: switch (条件) { case 値1: 処理1; ... }; NSチャート 条件 値1 値2 処理1 処理2 値n 処理n ... 前判定反復構造・繰り返しの前に条件を判定する・C言語: while (条件) 処理; 条件 真・偽 処理 偽・真 流れ図 NSチャート 条件 処理 条件が成している限り 処理を実する 後判定反復構造・繰り返しの後に条件を判定する・C言語: do 処理 while (条件); 条件 真・偽 処理 偽・真 流れ図 NSチャート 条件 処理 条件が成するまで 処理を実する カウンタ制御反復構造・制御変数(カウンタ変数ともいう)・制御変数の初期化処理・制御変数の繰り返し1回あたりの増加処理(減少処理)・制御変数の終了値を判定する終了条件・C言語: for (初期化処理; 終了条件; 増加処理) 処理; int i; for (i=0; i<10; i++) {} → While カウンタ制御反復構造 繰返条件 真 処理 流れ図 初期化処理 偽 増加処理 カウンタが 終了値に 達したかを判定する カウンタ制御反復構造 NSチャート 終了条件 処理 初期化処理 増加処理 NSチャート for I←1. N 処理 フローチャートの利点と欠点・アルゴリズムの理解に補助となる標準シンボル・おおまかなプログラムの流れを表現するには良いツール・プログラムの完全な仕様記述にフローチャートは向かない(時間と紙の無駄) NSチャートの利点と欠点・慣れれば、フローチャートより直感的に理解できる。・反復構造中の例外処理(break, continue)を上手く表現できない。 データフロー図・アルゴリズム以上のもの・データの流れを図式化する 外部データソース 処理プロセス 処理プロセス データストア データ名 データ名 データ名 データフロー図の例 顧客 受付 顧客名簿 注書 顧客照会 顧客情報 在庫 確認 メーカー発注書 商品 商品伝票 在庫 DB 在庫照会 在庫情報 出荷 商品 商品 出荷指書 フローチャート詳細 流れ線 端 START END 処理 判断 結合 A A 出 繰返し 磁気ディスク 定義済み処理 擬似言語とは・擬似言語・擬似コード、PDL(Program Description Language)とも呼ぶ。基本的な語彙や構文だけを約束しておき、細部については自由に誤解を生じない程度に表現。 擬似言語の基本・基本処理(入力・計算・出力)・入力 read 変数名・計算 変数名 ← 変数(数値) / 変数(数値)・出力 write 変数名・コンピュータ言語はよくBNFで記述される。擬似言語についてもBNFで記述してみましょう。本授業における記述形式の取り決め(Backus-Naur Form, BNF)・ ::= 定義・ <> 任意の表現を許す・ {} 0回以上の出現・ | あるいは・ 例えば・ 10以内の整数 <digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 バックス記法(BNF: Backus-Naur Form)・言語を定義するためのメタ言語・メタ言語 言語を記述する言語・メタデータ データを記述するデータ・Music data ?

【要点】

- ・ 制御構造とプログラムの図示 原田研究室 准教授 万 偉偉 基礎工学部 F522号室 wan@sys.
- ・ jp 復習 ・ アルゴリズムとは ・ 問題を解くための論理または手順 ・ プログラミングとは ・ データ構造とアルゴリズム ・ 良いプログラム、計算方法の選択 ・ 事前条件・事後条件・不変条件 ・ 自然言語・疑似コーディング・Cプログラム ・ データ型 ・ 文字・整数・実数・配列・構造体・共用体・ポインタ ・ データ構造 ・ リスト、待ち行列・スタック・木・ハッシュ表・グラフ ・ データ構造+操作の手法 → 抽象データ型 ・ 演算子 ・ 優先順位 本日の内容 ・ フローチャート ・ NSチャート ・ プログラムの制御構造 (構造化プログラミング) ・ 擬似言語 (擬似コード, PDL) ・ 拡張バックス記法 ・ ハイレベル 計算機言語の制御構造 ・ 接続構造 ・ 選択構造 ・ 単一選択構造 (if ~ then ~ end if) ・ 分岐選択構造 (if ~ then ~ else ~ end if) ・ 複数選択構造 (switch ~ end switch) ・ 反復構造 ・ 前判定反復構造 (while ~ end while) ・ 後判定反復構造 (do ~ until) ・ カウンタ制御反復構造 (for) 各制御構造の図示 ・ フローチャート (流れ図, Flow Chart) ・ 論文や説明書提案書の作成によく使われる ・ NSチャート (Nassi-Schneiderman Chart) 接続構造 ・ 処理 1, 処理 2...処理 n の逐次実行 ・ このようなまとまりをブロック (block) という. 処理 1 処理 2 処理 n 流れ図 処理 1 処理 2 処理 n NSチャート 単一選択構造 ・ 条件が真のときに処理を行う ・ C言語: if (条件) 処理; 流れ図 条件 真 NSチャート 条件 真 偽処理 処理 偽 分岐選択構造 ・ C言語: if (条件) then 処理 1; else 処理 2; 流れ図 NSチャート 条件 真 偽 処理 1 処理 2 条件 真 処理 1 偽 処理 2 複数選択構造 ・ C言語: switch (条件) { case 値 1: 処理 1; ... }; 流れ図 条件=値 2 真 処理 1 処理 2 真 条件=値 1 偽 条件=値 n 処理 n 真 偽 偽 複数選択構造 ・ C言語: switch (条件) { case 値 1: 処理 1; ... }; NSチャート 条件 値 1 値 2 処理 1 処理 2 値 n 処理 n ... 前判定反復構造 ・ 繰り返しの前に条件を判定する ・ C言語: while (条件) 処理; 条件 真 ・ 偽 処理 偽 ・ 真流れ図 NSチャート 条件 処理 条件が成している限り 処理を実する 後判定反復構造 ・ 繰り返しの後に条件を判定する ・ C言語: do 処理 while (条件); 条件 真 ・ 偽 処理 偽 ・ 真 流れ図 NSチャート 条件 処理 条件が成するまで 処理を実する カウンタ制御反復構造 ・ 制御変数 (カウンタ変数ともいう) ・ 制御変数の初期化処理 ・ 制御変数の繰り返し 1 回あたりの増加処理 (減少処理) ・ 制御変数の終了値を判定する終了条件 ・ C言語: for (初期化処理; 終了条件; 増加処理) 処理; ・ int i; ・ for (i = 0; i < 10; i++) {} → While カウンタ制御反復構造 繰返条件 真 処理 流れ図 初期化処理 偽 増加処理 カウンタが 終了値に 達したかを 判定する カウンタ制御反復構造 NSチャート 終了条件 処理 初期化処理 増加処理 NSチャート for I ← 1.
- ・ N 処理 フローチャートの利点と欠点 ・ アルゴリズムの理解に補助となる標準シンボル ・ おおまかなプログラムの流れを表現するには良いツール ・ プログラムの完全な仕様記述にフローチャート は向かない (時間と紙の無駄) NSチャートの利点と欠点 ・ 慣れれば, フローチャートより直感的に理解できる. ・ 反復構造中の例外処理 (break, continue) を上手く 表現できない. データフロー図 ・ アルゴリズム以上のもの ・ データの流れを図式化する 外部データソース 処理プロセス 処理プロセス データストア データ名 データ名 データ名 データフロー図の例 顧客 受付 顧客名簿 注書 顧客照会 顧客情報 在庫 確認 メーカ発注書 商品 商品伝票 在庫DB 在庫照会 在庫情報 出荷 商品 商品 出荷指書 フローチャート詳細 流れ線 端 START END 処理 判断 結合 A A 出 繰り返し 磁気ディスク 定義済み処理 擬似言語とは ・ 擬似言語 ・ 擬似コード, PDL (Program Description Language) と も呼ぶ. 基本的な語彙や構文だけを約束しておき, 細部については自由に誤解を生じない程度に表現. 擬似言語の基本 ・ 基本処理 (入力・計算・出力) ・ 入力 read 変数名 ・ 計算 変数名 ← 変数 (数値) / 変数 (数値) ・ 出力 write 変数名 ・ コンピュータ言語はよくBNFで記述される ・ 擬似言語についてもBNFで記述してみましょう ・ 本授業における記述形式の取り決め (Backus-Naur Form, BNF) ・ ::= 定義 ・ <> 任意の表現を許す ・ {} 0 回以上の出現 ・ | あるいは ・ 例えば ・ 10以内の整数 <digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 バックス記法 (BNF: Backus-Naur Form) ・ 言語を定義するためのメタ言語 ・ メタ言語 言語を記述する言語 ・ メタデータ データを記述するデータ ・ Music data ?
- ・ 音声 ・ 曲 metadata?

- ・ 作曲者・演奏者・プログラミング言語やデータ記述言語の文法決める 行う・表記（どのような文字を使うか）についての定義を行うためのものであり、演算や言語の機能まで定めるものではない バッカス記法（BNF: Backus-Naur Form）・ バッカス記法（BNF: Backus-Naur Form）・ $\langle X \rangle$ は、Xが表す記述を表現・ $::=$ は左辺の記述が右辺の記述で表現される・ $|$ はこの左右のいずれかの記述が可能・ $|$ で区切られていない複数の記述は、記述順に並べて表す・ $\langle \rangle$ でくくられていない表現は、その表現のまま記述（ダブルクォーテーションやシングルクォーテーションを使う流儀もある） バッカス記法の歴史・ John Backus and Peter Naurによる発明・ 最初の汎用プログラミング言語Algoal 58 / Algol 60 の文法を定義するために開発され・ Algoal 58/60 CPL BCPL B C・ Algoal: Algorithmic Language・ CPL: Combined Programming Language・ BCPL: Basic Combined Programming Language・ B・ C・ 最初の汎用プログラミング言語1958年、ACM提出 バッカス記法の例・ $\langle \text{assignment statement} \rangle ::= \langle \text{variable} \rangle = \langle \text{expression} \rangle$ ・ $\langle \text{number} \rangle ::= \langle \text{digit} \rangle | \langle \text{number} \rangle \langle \text{digit} \rangle$ ・ $\langle \text{digit} \rangle ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$ ・ $\langle \text{number} \rangle ::= \langle \text{digit} \rangle | \langle \text{digit_positive} \rangle \langle \text{digit_string} \rangle$ ・ $\langle \text{digit_positive} \rangle ::= 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9$ ・ $\langle \text{digit_string} \rangle ::= \langle \text{digit} \rangle | \langle \text{digit} \rangle \langle \text{digit_string} \rangle$ 設問 バッカス記法の例 再帰的な定義・ $\langle \text{二進数} \rangle ::= ?$

【重要語句】

- ・ 処理
- ・ 条件
- ・ チャート
- ・ while
- ・ 流れ図
- ・ 言語
- ・ digit
- ・ then
- ・ カウンタ制御反復構造
- ・ 変数の名前

【復習チェックリスト】

- ☐ 処理 を説明できる
- ☐ 条件 を説明できる
- ☐ チャート を説明できる
- ☐ while を説明できる
- ☐ 流れ図 を説明できる